Workshop: Starting the project

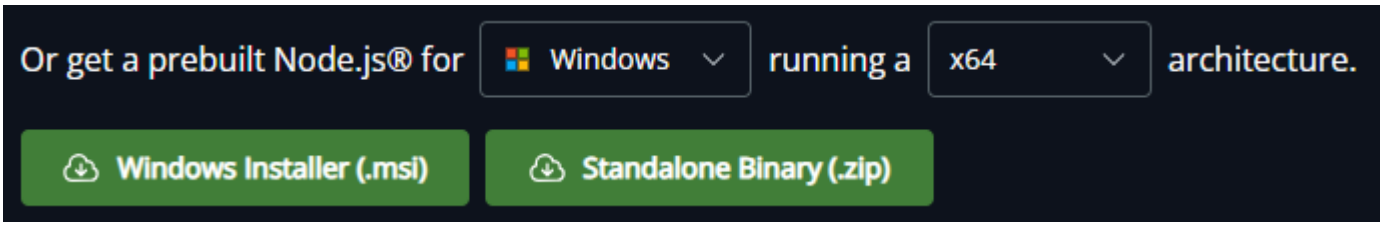
This workshop is dedicated to starting the project. During this sessions, you will work in a group of 4-5 students. You need one computer per student. The idea is to discuss your code within the group. You can create help the other members of the group to setup their computer to start the project. I recall that the project is made **INDIVIDUALLY or WITH A GROUP of TWO STUDENTS**.

1) Installing Node.js - duration: 15 minutes:

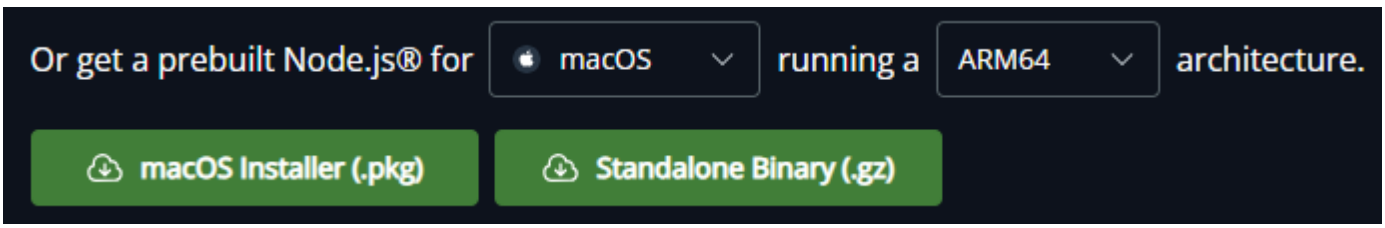
You must know if you have node installed on your computer: run the command “node -v” in a terminal. If you have node v22.19.0, perfect, node is installed on your system!

If not, please, download and install Node.js on your computer, please use the “Long Term Support (LTS)” version from: https://nodejs.org/en/download

For PC, choose the correct processor and download the .msi installer:



For Mac, choose the right processor x64 (Intel) or ARM64 (M1, M2, …) and download the .pkg installer:



Install node by running this installer…

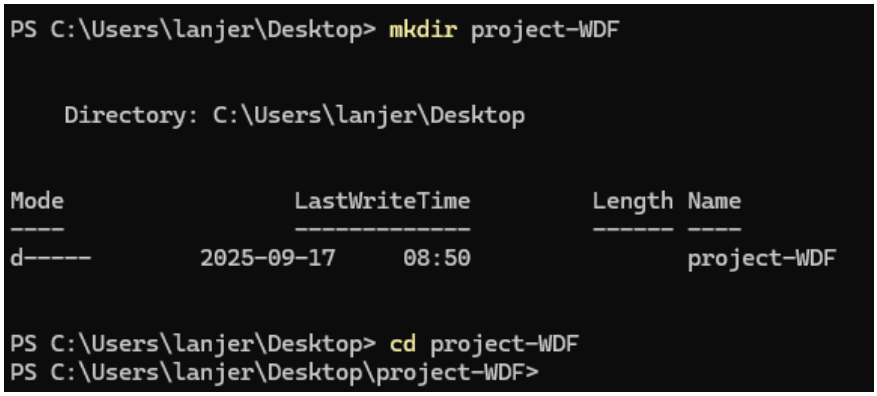
Check the installation from a terminal, “node -v” must give you “v22.19.0”.

```
PS C:\> node -v
v22.19.0
PS C:\>
```

If it works, congratulations/grattis, Node is installed on your system!

2) The project folder (or directory) – 10 minutes

Create a folder wherever you want on your system, **BUT** you must be able to locate this folder and go into it from the terminal! A good idea is to put it on your Desktop (easy to reach from the terminal…).



On Windows/Linux/MacOS, the commands are:

cd Desktop

mkdir project-WDF

cd project-WDF

You can list the files and folders in this directory using “dir” on Windows or “ls -l” on MacOS/Linux.

You should only have the special folders “.” and “..” for now. These folders are: “.”, the current directory and “..”, the parent directory of the current directory.

The current directory appears on the Windows prompt:

To know the current directory on MacOS/Linux: just type “pwd” (print working directory) and press enter in a terminal.

3) Initializing the project - duration: 10 minutes

While in the project directory, in the terminal, type “npm init”. Please answer the questions on the command line with your own information:

```
PS C:\> cd .\Users\lanjer\Desktop\project-WDF\
PS C:\Users\lanjer\Desktop\project-WDF> npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See `npm help init` for definitive documentation on these fields
and exactly what they do.

Use `npm install <pkg>` afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.
package name: (project-wdf)
version: (1.0.0)
description: My Web Dev Fun project.
entry point: (index.js) server.js
test command:
git repository:
keywords: Project, Node, Express, handlebars, SQLite3, Web Dev Fun, Jönköping University.
author: JL
license: (ISC)
type: (commonjs)
```

```
About to write to C:\Users\lanjer\Desktop\project-WDF\package.json:

{
  "name": "project-wdf",
  "version": "1.0.0",
  "description": "My Web Dev Fun project.",
  "main": "server.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [
    "Project",
    "Node",
    "Express",
    "handlebars",
    "SQLite3",
    "Web",
    "Dev",
    "Fun",
    "Jönköping",
    "University."
  ],
  "author": "JL",
  "license": "ISC",
  "type": "commonjs"
}


Is this OK? (yes)

PS C:\Users\lanjer\Desktop\project-WDF>
```

“npm init” will create the file “package.json” file which will contain the information about your project.

I recommend you to name the main file “server.js”, for the other information, you can just press “Enter” if you do not want to worry about them. You finish by pressing “y” then “Enter” at the final question: “Is this OK? (yes)”.

The initialization file “package.json” is just a text file containing a JSON variable (we will see in a coming lecture what it is). It means you can modify it by hand afterward, so no worries about the content of this file for now. It should look like this (in the terminal, “type package.json” on Windows or “cat package.json” on MacOS/Linux):

```
PS C:\Users\lanjer\Desktop\project-WDF> type package.json
{
  "name": "project-wdf",
  "version": "1.0.0",
  "description": "My Web Dev Fun project.",
  "keywords": [
    "Project",
    "Node",
    "Express",
    "handlebars",
    "SQLite3",
    "Web",
    "Dev",
    "Fun",
    "JÃ¶nkÃ¶ping",
    "University."
  ],
  "license": "ISC",
  "author": "JL",
  "type": "commonjs",
  "main": "server.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  }
}
PS C:\Users\lanjer\Desktop\project-WDF>
```

Create a “views” and a “public” folder. In “views”, we will put our HTML pages, in “public”, we will put our CSS, JS, image files.

```
PS C:\Users\lanjer\Desktop\project-WDF> mkdir views
```

```
PS C:\Users\lanjer\Desktop\project-WDF> mkdir public
```

```
PS C:\Users\lanjer\Desktop\project-WDF> dir


    Directory: C:\Users\lanjer\Desktop\project-WDF


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        2025-09-17     09:18                public
d-----        2025-09-17     09:18                views
-a----        2025-09-17     09:16            421 package.json


PS C:\Users\lanjer\Desktop\project-WDF>
```

Install “express” and “sqlite3” packages using npm:

```
PS C:\Users\lanjer\Desktop\project-WDF> npm install express sqlite3
npm warn deprecated @npmcli/move-file@1.1.2: This functionality has been moved to @npmcli/fs
npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Check out lru-cache i
f you want a good and tested way to coalesce async requests by a key value, which is much more comprehensive and powerfu
l.
npm warn deprecated npmlog@6.0.2: This package is no longer supported.
npm warn deprecated rimraf@3.0.2: Rimraf versions prior to v4 are no longer supported
npm warn deprecated glob@7.2.3: Glob versions prior to v9 are no longer supported
npm warn deprecated are-we-there-yet@3.0.1: This package is no longer supported.
npm warn deprecated gauge@4.0.4: This package is no longer supported.

added 181 packages, and audited 182 packages in 5s

27 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\lanjer\Desktop\project-WDF>
```

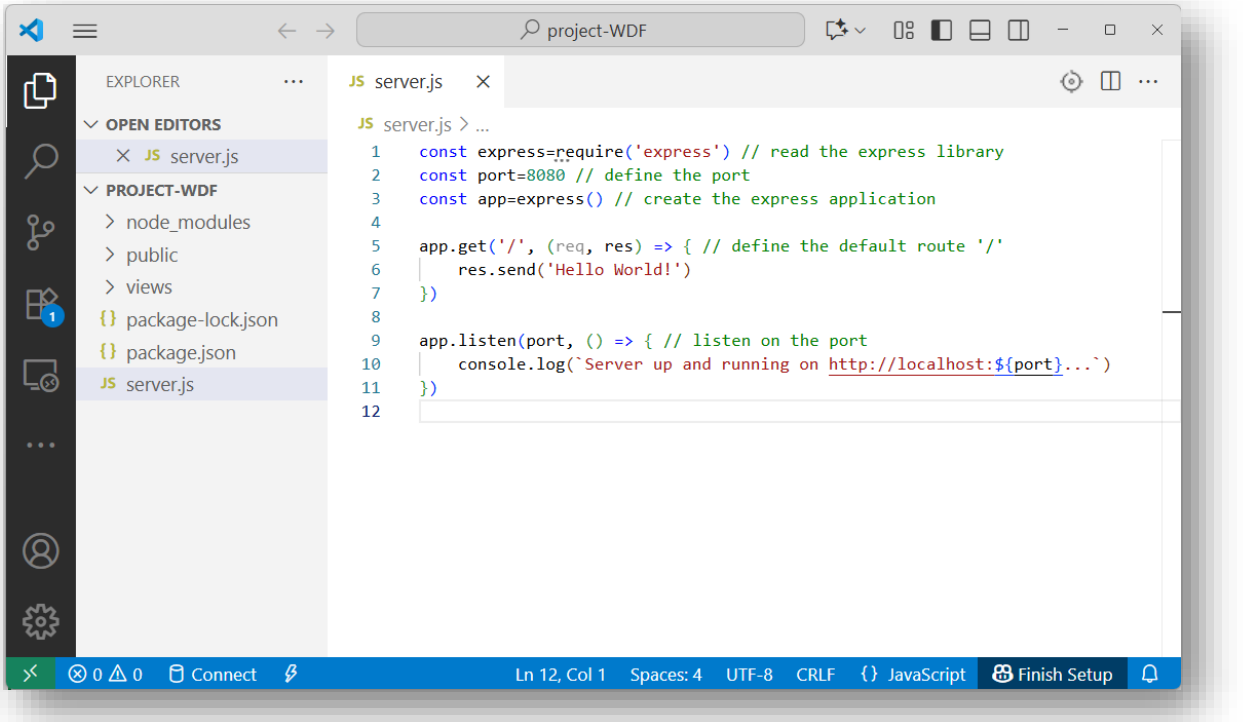
```
PS C:\Users\lanjer\Desktop\project-WDF> dir


    Directory: C:\Users\lanjer\Desktop\project-WDF


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         2025-09-17     09:21                node_modules
d-----         2025-09-17     09:18                public
d-----         2025-09-17     09:18                views
-a----         2025-09-17     09:21          78446 package-lock.json
-a----         2025-09-17     09:21            495 package.json


PS C:\Users\lanjer\Desktop\project-WDF>
```

Now, let's start coding using Visual Studio Code! `PS C:\Users\lanjer\Desktop\project-WDF> code .`

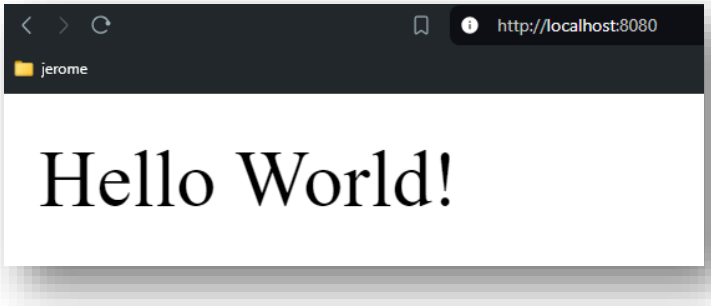
4) Coding in Node.js - duration: 1 hour

Please install Visual Studio Code on your computer. It is a free and powerful code editor!

Open the folder "Desktop/project-WDF", create a new file server.js with the following code:



In Visual studio code, you can open a terminal directly at the bottom of the page. Do it and run your server: "node server.js", open your preferred browser and open the address: http://localhost:8080:



Congratulations, it works!

Now, we want to serve, and to send a full web page back to the client.

I used the CV created in lab 1 as an example. Copy a web page (for me, “cv-jl.html”), the associated CSS file (for me, “styles-cv.css”) and the images (for me “jerome3.png”, plus two SVG files) into your project folder:

```
PS C:\Users\lanjer\Desktop\project-WDF> dir


    Directory: C:\Users\lanjer\Desktop\project-WDF


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        2025-09-17     09:21                node_modules
d-----        2025-09-17     09:18                public
d-----        2025-09-17     09:18                views
-a----        2024-09-02     13:44           7852 25777.svg
-a----        2024-09-02     13:31          31077 33420.svg
-a----        2025-09-17     09:32           6320 cv-jl.html
-a----        2024-08-23     16:34         278750 jerome3.png
-a----        2025-09-17     09:21          78446 package-lock.json
-a----        2025-09-17     09:21            495 package.json
-a----        2025-09-17     09:27            379 server.js
-a----        2024-09-03     10:54           2244 styles-cv.css


PS C:\Users\lanjer\Desktop\project-WDF>
```

Add a new route “/cv” into your server.js file to send the HTML page (for me “cv-jl.html”) back.

```
app.get('/cv', function (req, res) {
    res.sendFile('/cv-jl.html')
})
```

Stop your running server (CTRL+C in the terminal) and re-run it. Test the “/” route and the “/cv” route.

… It doesn’t work.

a) why?
b) Try to find a solution on the Internet to this problem (copy and paste the error message on your favorite search engine)…

Stop your running server (CTRL+C in the terminal) and re-run it. Test the “/” route and the “/cv” route.

… It does work; however, the style does not apply and the image does not show up.

c) Why?
d) Try to find a solution on the Internet to this problem (copy and paste the error message on your favorite search engine)…

… One simple solution is to add two new routes. Try it!

5) **Organizing the folders - duration: 15 minutes**

When your application will contain many pages, putting everything into the main directory is not a good idea… Let's organize the files into different directories…

Move the PNG image into "public/img/" folder, move the cv HTML file into "views/", move the CSS file into "public/css/".

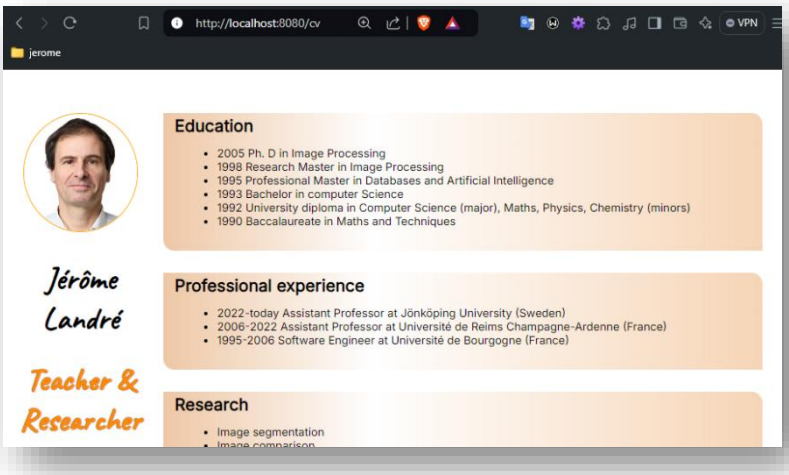
Add the public folder as a path to your application:

```
app.use(express.static('public'))
```

Re-run and test. If it is not working, perhaps you must change the code in your html file:

```
<link rel="stylesheet" href="/css/styles-cv.css">
<img class="photoProfile" src="/img/jerome3.png" alt="photoJL" />
```

Re-run and retry, it should work!



6) **SQLite3 - duration: 30 minutes**

Adding the SQLite3 database to your project directory is mandatory because we want to build a dynamic website with data extracted from the database.

We already installed "sqlite3" package into your project directory with express above. However, what file/directory changes during this installation of a npm package? Why? What is the total size of your project directory? Can you do something to reduce this size?

We will add the code to create a database file, a table and populate it with data:

```
const sqlite3=require('sqlite3') // read the sqlite3 package
```

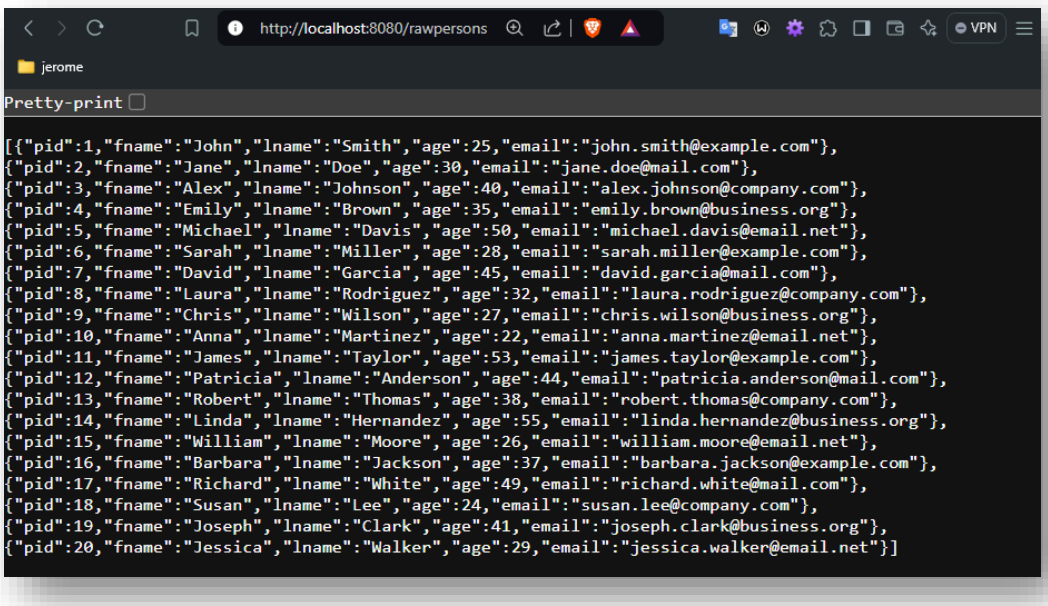
```
const dbFile='my-project-data.sqlite3.db'
db = new sqlite3.Database(dbFile)

// creates table projects at startup
db.run("CREATE TABLE Person (pid INTEGER PRIMARY KEY, fname TEXT NOT NULL, lname TEXT NOT NULL, age INTEGER, email TEXT)", (error) => {
    if (error) {
      // tests error: display error
      console.log("---> ERROR: ", error)
    } else {
      // tests error: no error, the table has been created
      console.log("---> Table created!")
      db.run("INSERT INTO Person (fname, lname, age, email) VALUES ('John', 'Smith', 25, 'john.smith@example.com'), ('Jane', 'Doe', 30, 'jane.doe@mail.
      com'), ('Alex', 'Johnson', 40, 'alex.johnson@company.com'), ('Emily', 'Brown', 35, 'emily.brown@business.org'), ('Michael', 'Davis', 50, 'michael.
      davis@email.net'), ('Sarah', 'Miller', 28, 'sarah.miller@example.com'), ('David', 'Garcia', 45, 'david.garcia@mail.com'), ('Laura', 'Rodriguez', 32,
      'laura.rodriguez@company.com'), ('Chris', 'Wilson', 27, 'chris.wilson@business.org'), ('Anna', 'Martinez', 22, 'anna.martinez@email.net'), ('James',
      'Taylor', 53, 'james.taylor@example.com'), ('Patricia', 'Anderson', 44, 'patricia.anderson@mail.com'), ('Robert', 'Thomas', 38, 'robert.thomas@company.
      com'), ('Linda', 'Hernandez', 55, 'linda.hernandez@business.org'), ('William', 'Moore', 26, 'william.moore@email.net'), ('Barbara', 'Jackson', 37,
      'barbara.jackson@example.com'), ('Richard', 'White', 49, 'richard.white@mail.com'), ('Susan', 'Lee', 24, 'susan.lee@company.com'), ('Joseph', 'Clark',
      41, 'joseph.clark@business.org'), ('Jessica', 'Walker', 29, 'jessica.walker@email.net');" , function (err) {
        if (err) {
            console.log(err.message)
        } else {
            console.log('---> Rows inserted in the table Person.')
        }
      })
    }
})
```

Because this code is tricky, I created a file on Canvas for you… :)

Now, create a new route “/rawpersons” with the code to get information raw data from the table and send it back to the client (raw):

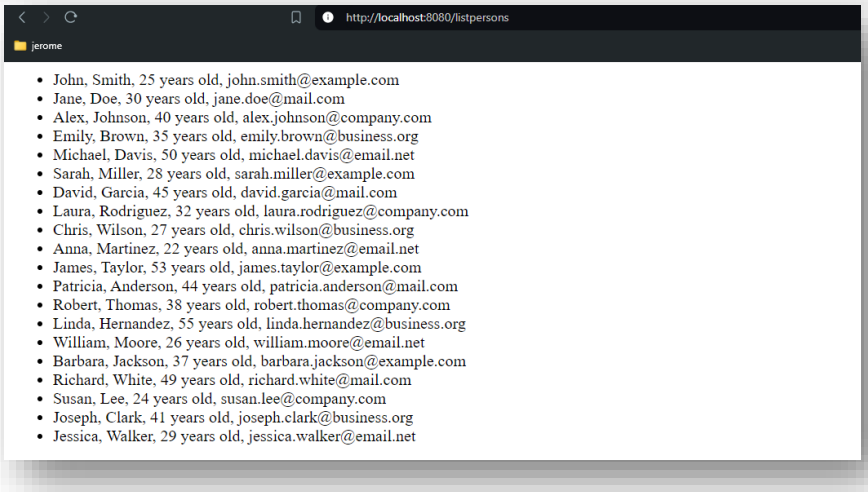
```
app.get('/rawpersons', function (req, res) {
    db.all('SELECT * FROM Person', function (err, rawPersons) {
        if (err) {
            console.log('Error: '+err)
        } else {
            console.log('Data found, sending it back to the client...')
            res.send(rawPersons)
        }
    })
})
```

Re-run the server and test the result:



Please complete the following code to create a route “/listpersons” which will display the result as a HTML list (tags <ul> and <li>):

```
app.get('/listpersons', function (req, res) {
    db.all('SELECT * FROM Person', function (err, rawPersons) {
        if (err) {
            console.log('Error: '+err)
        } else {
            listPersonsHTML='<ul>'
            rawPersons.forEach( function (onePerson) {
                listPersonsHTML+='<li>'
                listPersonsHTML+=`${onePerson.fname}, ` //be careful: backticks!
                listPersonsHTML+=
                listPersonsHTML+=
                listPersonsHTML+=
                listPersonsHTML+='</li>'
            })
            listPersonsHTML+='</ul>'
            //console.log(listPersonsHTML)
            res.send(listPersonsHTML)
        }
    })
})
```



This part is tricky but once you have done it for your project, you won’t change it, except for small improvements. :)

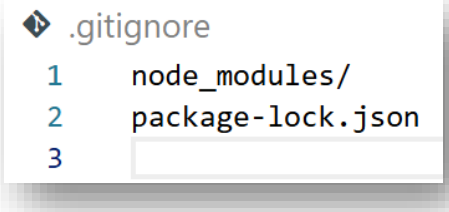
7) Adding Github to your project folder

You MUST link your project folder to Github, it makes sense if your are two users doing the project together. It also makes sense if you are alone because it will guarantee that you have a backup of your project on the cloud!

To create this link, please follow the instructions given in the workshop about Javascript and git that I previously published on Canvas.

Because we don’t need “node_modules/” and “package-lock.json” in our project directory (they can be added to our project using “npm -l”), we must create a “.gitignore” file to specify which files/folders must be ignored when pushing to or pulling from Github.

Add a “.gitignore” file in your project directory:



8) Thinking about YOUR project

Now that you know how to start your project, please think of the website you want to build.

You can discuss with your colleagues the tables you can put in it, the fields, the links between these tables…

I recall that the project is **yours** (individual project or group of two students), you can make your own portfolio with a list of projects you made and the skills they require to get two tables: project and skill, the website of a company with persons and computers for instance, the website of a music band, the website of a bakery… The list of possibilities is endless.

However, please remember the **KISS** paradigm: **Keep It Super Simple**! If you plan for grade 3, having only two or three tables (please look at the requirements) is sufficient. Multiplying the number of tables will increase the work and code to write! Think about it…

Ask for help and feedback during the labs!

OK, it was **not EASY**, I know.

“But using Node/Express/SQLite3 is the same as swimming or biking: once you know it, it is forever, it is always the same...”

That is why you **MUST go to the labs** to **ask questions** if you have problems with your code. The lab assistants will **help** you and give you **feedback** on your code! :)

TACK OCH GRATTIS FÖR IDAG!